SLOT 5

STREAMBUILDER – REPOSITORY PATTERN – REACTIVE THINKING

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

- StreamBuilder in Flutter
- Repository Pattern
- Reactive Patterns (Concept)

Slot 5 là **bước chuyển tư duy quan trọng** từ:
Future (bất đồng bộ 1 lần)
→ Stream (dữ liệu thay đổi liên tục – realtime)
→ UI phản ứng theo dữ liệu (Reactive UI)
Sinh viên không chỉ "viết được code", mà **bắt đầu hiểu cách dữ liệu chảy trong app Flutter thực tế**.

I. STREAM & STREAMBUILDER

◈ Khái niệm cốt lõi bắt buộc phải hiểu

- **Stream là luồng dữ liệu phát ra nhiều lần theo thời gian**, không phải chỉ 1 kết quả như Future
- **StreamBuilder là Widget lắng nghe Stream và tự động rebuild UI khi có dữ liệu mới**
- UI **không chủ động gọi lại dữ liệu**, mà **phản ứng khi dữ liệu thay đổi**

🎯 Mục tiêu học tập – sau bài này sinh viên làm được gì

- Phân biệt rõ **Future vs Stream** trong Flutter
- Dùng **StreamBuilder** để:
  - Hiển thị dữ liệu realtime (counter, timer, message, trạng thái)
  - Xử lý các trạng thái: loading – có dữ liệu – lỗi
- Hiểu được **vì sao Stream phù hợp cho app realtime** (chat, notification, sensor, live data)

📘 Ý nghĩa sư phạm – vì sao phải học

- Giúp sinh viên **thoát khỏi tư duy "gọi API → xong"**
- Đặt nền móng cho **realtime app** và các kiến trúc nâng cao (BLoC, Riverpod…)

II. REPOSITORY PATTERN (TÁCH NGUỒN DỮ LIỆU KHỎI UI)

◈ Khái niệm cốt lõi bắt buộc phải hiểu

- **Repository là lớp trung gian**:
  - UI **không biết dữ liệu đến từ đâu**
  - Chỉ biết: "Tôi cần dữ liệu"
- Repository chịu trách nhiệm:
  - Lấy dữ liệu (API, database, mock, stream)
  - Trả dữ liệu về cho UI dưới dạng **Future hoặc Stream**

🎯 Mục tiêu học tập – sau bài này sinh viên làm được gì

- Biết cách **không viết logic lấy dữ liệu trực tiếp trong Widget**
- Tạo Repository đơn giản trả về:
  - `Future<T>` (ôn lại)
  - `Stream<T>` (trọng tâm Slot 5)
- Dễ dàng:
  - Thay đổi nguồn dữ liệu
  - Test logic
  - Mở rộng dự án sau này

📘 Ý nghĩa sư phạm – vì sao phải học

- Giúp sinh viên **viết code có tổ chức**, không "dồn hết vào UI"
- Là bước đệm để tiếp cận:
  - Clean Architecture

PhucPT10@fpt.edu.vn

    - BLoC / MVVM / Riverpod
- Phù hợp tư duy **dự án thật – làm việc nhóm**

III. REACTIVE THINKING – TƯ DUY UI PHẢN ỨNG THEO DỮ LIỆU

◈ Khái niệm cốt lõi bắt buộc phải hiểu

- **Reactive = UI không điều khiển dữ liệu, UI chỉ phản ứng**
- Dữ liệu thay đổi → Stream phát giá trị mới → UI tự cập nhật
- Không cần `setState()` cho mọi thứ

🎯 Mục tiêu học tập – sau bài này sinh viên làm được gì

- Hiểu cách **dữ liệu "chảy" trong Flutter app**
- Thiết kế UI theo câu hỏi:
  "Nếu dữ liệu thay đổi thì UI sẽ phản ứng thế nào?"
- Nhận diện được các tình huống:
    - Dùng Future là đủ
    - Khi nào **bắt buộc phải dùng Stream**

📘 Ý nghĩa sư phạm – vì sao phải học

- Giúp sinh viên **đổi tư duy từ imperative → reactive**
- Chuẩn bị tâm thế cho:
    - State management
    - App lớn, realtime, nhiều màn hình
- Tránh lỗi phổ biến: **UI rối – logic lặp – khó mở rộng**

🗂 NGUỒN TÀI LIỆU THAM KHẢO UY TÍN (DỄ TIẾP CẬN)

1. **Dart – Asynchronous Programming (Stream)**
    - https://dart.dev/tutorials/language/streams
2. **Flutter – StreamBuilder Widget (Official Docs)**
    - https://docs.flutter.dev/cookbook/networking/streaming-data
3. **Flutter YouTube – Async & Reactive UI**
    - Flutter Official Channel (video dễ hiểu cho người mới)
4. **Reso Coder – Streams & Reactive Thinking**
    - Bài giảng giải thích chậm, nhiều ví dụ thực tế
5. **Flutter Architecture Samples**
    - https://github.com/flutter/samples
      (tham khảo cách tách UI – data)

PhucPT10@fpt.edu.vn

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

I. STREAM & STREAMBUILDER – DÒNG DỮ LIỆU LIÊN TỤC

◈ Khái niệm quan trọng (BẮT BUỘC HIỂU ĐÚNG)

- **Stream KHÔNG PHẢI là Future chạy nhiều lần**
  - ❌ Hiểu sai phổ biến: "Stream = Future nhưng emit nhiều lần"
  - ✅ Bản chất đúng:
    - **Future**: trả về **1 giá trị → kết thúc**
    - **Stream**: phát **nhiều giá trị theo thời gian**
- **Stream đại diện cho dữ liệu có thể thay đổi liên tục**
  - Ví dụ: counter realtime, chat message, notification, sensor
- **StreamBuilder là cầu nối giữa Stream và UI**
  - UI **không gọi dữ liệu**
  - UI **chỉ lắng nghe và phản ứng**

👉 So sánh tư duy:

- Future: *"Khi nào xong thì trả kết quả cho tôi"*
- Stream: *"Mỗi lần dữ liệu đổi, hãy báo cho tôi"*

◈ Kỹ thuật / thao tác PHẢI THÀNH THẠO

- Biết dùng **StreamBuilder đúng cách**
  - Hiểu rõ:
    - `snapshot.connectionState`
    - `snapshot.hasData`
    - `snapshot.hasError`
- Biết rằng:
  - Stream **có thể phát dữ liệu trước – trong – sau**
- Biết:
  - **StreamBuilder tự rebuild UI khi có dữ liệu mới**
  - Không cần `setState()` cho dữ liệu stream

◈ Lỗi thường gặp của sinh viên mới

- ❌ **Code không lỗi nhưng UI không update**
  - Nguyên nhân:
    - Stream không phát dữ liệu mới
    - Dùng Future thay cho Stream
- ❌ **Tạo Stream trong `build()`**
  - Nguyên nhân:
    - Không hiểu vòng đời widget
    - Mỗi lần rebuild → tạo Stream mới
- ❌ **Nhầm Stream với vòng lặp**
  - Nguyên nhân:
    - Tư duy đồng bộ (for, while)

◈ Best Practices (Áp dụng NGAY)

- ✅ Stream **phải được tạo bên ngoài UI**
- ✅ UI **chỉ dùng StreamBuilder để lắng nghe**
- ✅ Mỗi Stream = **1 nguồn dữ liệu rõ ràng**
- ❌ Không xử lý logic trong `builder`

🎓 Ý nghĩa sư phạm

- Đây là **cửa ngõ đầu tiên vào realtime app**

PhucPT10@fpt.edu.vn

- Nếu không nắm Stream:
  - Chat, notification, feed realtime → **rối hoặc sai hoàn toàn**
- Là nền tảng bắt buộc cho:
  - BLoC
  - RxDart
  - State management nâng cao

II. REPOSITORY PATTERN – TÁCH DỮ LIỆU KHỎI UI

◈ Khái niệm quan trọng (BẮT BUỘC HIỂU ĐÚNG)

- **Repository không phải là "file cho có"**
- Repository là:
  - **Nguồn dữ liệu duy nhất**
  - UI **không biết dữ liệu đến từ đâu**
- UI **chỉ quan tâm dữ liệu**, không quan tâm:
  - API
  - Database
  - Stream hay Future bên trong

👉 So sánh tư duy:

- ❌ UI gọi API trực tiếp
- ✅ UI gọi Repository → Repository xử lý tất cả

◈ Kỹ thuật / thao tác PHẢI THÀNH THẠO

- Biết tạo Repository trả về:
  - `Future<T>`
  - `Stream<T>`
- Biết inject Repository vào Widget (constructor đơn giản)
- Biết:
  - UI **chỉ gọi repository**
  - Không gọi API / service trong Widget

◈ Lỗi thường gặp của sinh viên mới

- ❌ **Không hiểu vì sao phải dùng Repository**
  - Nguyên nhân:
    - App nhỏ → chưa thấy vấn đề
- ❌ **Nhét logic lấy dữ liệu vào Widget**
  - Nguyên nhân:
    - Tư duy "viết cho chạy"
- ❌ **Mỗi màn hình tự fetch dữ liệu**
  - Nguyên nhân:
    - Không có nguồn dữ liệu trung tâm

◈ Best Practices (Áp dụng NGAY)

- ✅ Mỗi loại dữ liệu → 1 Repository
- ✅ UI không biết:
  - API endpoint
  - database
- ✅ Repository là nơi:
  - Quyết định dùng Future hay Stream

🎓 Ý nghĩa sư phạm

- Repository xuất hiện **TRƯỚC State Management**

PhucPT10@fpt.edu.vn

- Nếu không học Repository:
  - Khi học BLoC / Riverpod → **không hiểu luồng**
- Đây là nền móng của:
  - Clean Architecture
  - App lớn – làm việc nhóm

III. REACTIVE THINKING – TƯ DUY UI PHẢN ỨNG

◈ Khái niệm quan trọng (BẮT BUỘC HIỂU ĐÚNG)

- **Reactive = UI không điều khiển dữ liệu**
- UI **phản ứng khi dữ liệu thay đổi**
- Dữ liệu là trung tâm, UI chỉ là "người xem"

👉 So sánh:

- ❌ UI chủ động gọi → cập nhật → setState
- ✅ Dữ liệu thay đổi → UI tự cập nhật

◈ Kỹ thuật / thao tác PHẢI THÀNH THẠO

- Biết thiết kế UI theo câu hỏi:
  "Nếu dữ liệu đổi thì UI sẽ ra sao?"
- Biết:
  - Khi nào cần Stream
  - Khi nào Future là đủ
- Biết đọc luồng:
  - Data → Stream → UI

◈ Lỗi thường gặp của sinh viên mới

- ❌ **Dùng setState cho mọi thứ**
  - Nguyên nhân:
    - Chưa thoát tư duy imperative
- ❌ **Stream chạy nhưng UI rebuild loạn**
  - Nguyên nhân:
    - Stream phát quá nhiều
    - Không kiểm soát dữ liệu
- ❌ **Không hiểu UI rebuild khi nào**

◈ Best Practices (Áp dụng NGAY)

- ✅ UI càng "ngu" càng tốt
- ✅ Logic nằm ở:
  - Repository
  - Stream
- ❌ Không trộn:
  - UI + xử lý dữ liệu

🎓 Ý nghĩa sư phạm

- Reactive Thinking là **linh hồn của Flutter hiện đại**
- Nếu không học tư duy này:
  - App lớn → khó bảo trì
  - State management → học rất mệt
- Là nền tảng trực tiếp cho:
  - BLoC
  - Riverpod
  - MVVM

PhucPT10@fpt.edu.vn

🎯 TỔNG KẾT MỤC TIÊU SLOT 5

Sau Slot 5, sinh viên **BẮT BUỘC PHẢI**:

- ✅ Phân biệt rõ:
  - Future ≠ Stream
- ✅ Hiểu:
  - Vì sao UI hiện đại cần Reactive
- ✅ Làm được:
  - UI lắng nghe Stream qua StreamBuilder
  - Lấy dữ liệu qua Repository
- ✅ Sẵn sàng học:
  - BLoC / Riverpod / Rx (ở slot sau)

PhucPT10@fpt.edu.vn

III. ỨNG DỤNG THỰC TIỄN

👉 Trọng tâm:
Dữ liệu thay đổi → Stream phát → UI tự cập nhật
UI **không điều khiển**, UI **phản ứng**

I. VAI TRÒ CỦA SLOT 5 TRONG DỰ ÁN FLUTTER THỰC TẾ

🔄 Xử lý dữ liệu thay đổi liên tục (realtime)

Trong app Flutter thật, **rất nhiều dữ liệu không đứng yên**:

- Số lượng sản phẩm trong giỏ hàng
- Tin nhắn chat mới
- Trạng thái đơn hàng
- Thông báo hệ thống
- Counter / timer / progress

👉 **Future KHÔNG đủ**, vì:

- Future chỉ trả về **1 lần**
- Dữ liệu đổi → phải gọi lại → UI rối

👉 **Stream phù hợp**, vì:

- Dữ liệu đổi → Stream phát → UI tự cập nhật
- Không cần gọi lại thủ công

🧠 Tách UI – logic – data (để app không "toang")

Trong dự án thật:

- UI thay đổi liên tục
- Logic ngày càng phức tạp
- Nguồn dữ liệu có thể đổi (API → local → mock)

👉 **Repository Pattern** giúp:

- UI **không biết dữ liệu đến từ đâu**
- Logic lấy dữ liệu **tập trung một chỗ**
- Dễ test – dễ thay đổi

⚡ UI cập nhật tự động – không gọi `setState()` tràn lan

Slot 5 giúp chuyển từ:

- ❌ `setState()` cho mọi thay đổi
  sang
- ✅ **Reactive UI**:
  - UI lắng nghe Stream
  - Không cần "nhớ phải cập nhật UI"

👉 Đây là **bước chuyển tư duy quan trọng nhất**.

II. CÁC TÌNH HUỐNG THỰC TẾ (REAL-WORLD SCENARIOS)

🟢 TÌNH HUỐNG 1: APP CHAT ĐƠN GIẢN

🔷 Bối cảnh thực tế

- App chat sinh viên
- Tin nhắn mới đến liên tục
- UI phải cập nhật ngay khi có tin mới

🔷 Vấn đề gặp phải (nếu dùng cách cũ)

- Dùng `FutureBuilder`:
  - Chỉ load tin nhắn **1 lần**
- Dùng `setState()`:

PhucPT10@fpt.edu.vn

    - Gọi liên tục → code rối
- Logic fetch tin nhắn nằm trong Widget

◈ Cách áp dụng kỹ thuật Slot 5

- **Repository**
    - Quản lý Stream tin nhắn
- **StreamController**
    - Push tin nhắn mới vào Stream
- **StreamBuilder**
    - UI lắng nghe danh sách tin nhắn

👉 UI **KHÔNG**:

- fetch data
- xử lý logic

👉 UI **CHỈ**:

- render danh sách khi Stream emit

◈ Mục tiêu đạt được

- UI cập nhật realtime
- Code dễ đọc
- Dễ thêm:
    - typing indicator
    - seen status

◍ TÌNH HUỐNG 2: GIỎ HÀNG (CART) TRONG APP BÁN HÀNG

◈ Bối cảnh thực tế

- User thêm / xóa sản phẩm
- Tổng tiền & số lượng phải cập nhật ngay

◈ Vấn đề gặp phải

- Gọi `setState()` ở nhiều nơi
- Logic cộng tiền nằm trong Widget
- Mỗi màn hình quản lý cart một kiểu

◈ Cách áp dụng kỹ thuật Slot 5

- **Repository**
    - Là "nguồn chân lý" của cart
- Cart được expose dưới dạng **Stream**
- **StreamBuilder**:
    - Lắng nghe cart
    - UI tự cập nhật tổng tiền

👉 Khi cart đổi:

- Không cần gọi UI
- Stream phát → UI đổi

◈ Mục tiêu đạt được

- Tránh lỗi sai tiền
- Logic tập trung
- Dễ mở rộng:
    - voucher
    - shipping fee

PhucPT10@fpt.edu.vn

🟢 TÌNH HUỐNG 3: DASHBOARD / APP NỘI BỘ

🔷 Bối cảnh thực tế

- Dashboard hiển thị:
  - số user online
  - trạng thái hệ thống
- Dữ liệu thay đổi liên tục

🔷 Vấn đề gặp phải

- Polling bằng Future → delay
- UI không đồng bộ
- Khó test

🔷 Cách áp dụng kỹ thuật Slot 5

- Repository quản lý Stream dữ liệu
- Có thể fake / mock dữ liệu khi test
- UI chỉ render theo Stream

🔷 Mục tiêu đạt được

- UI luôn phản ánh đúng trạng thái
- Dễ thay nguồn dữ liệu
- App ổn định hơn

III. CASE STUDY – APP THEO DÕI TRẠNG THÁI ĐƠN HÀNG (REACTIVE)

📍 Bối cảnh dự án

- App theo dõi đơn hàng
- Trạng thái:
  - pending → shipping → completed

❗ Vấn đề ban đầu

- Dùng `setState()` cập nhật trạng thái
- Logic nằm trong Widget
- UI update không đồng bộ

🛠 Cách áp dụng Slot 5

- **Repository**
  - Quản lý Stream<OrderStatus>
- **StreamController**
  - Emit trạng thái mới
- **StreamBuilder**
  - UI hiển thị trạng thái hiện tại

👉 UI **không biết**:

- khi nào trạng thái đổi
- đổi bằng cách nào

✅ Kết quả

- UI tự cập nhật
- Code rõ ràng
- Dễ mở rộng:
  - thêm trạng thái mới
  - thêm animation

PhucPT10@fpt.edu.vn

IV. BÀI HỌC RÚT RA – REACTIVE THINKING

🔑 Điều quan trọng nhất

- **Dữ liệu là nguồn chân lý**
- UI chỉ là người "xem dữ liệu"

❌ Sai lầm sinh viên RẤT HAY MẮC

- Tạo Stream trong `build()`
- Không `dispose()` StreamController
- Dùng Stream chỉ để "cho giống bài mẫu"
- Trộn logic vào UI

✅ Nguyên tắc cần duy trì lâu dài

- UI càng đơn giản càng tốt
- Repository là trung tâm dữ liệu
- Stream dùng cho dữ liệu thay đổi liên tục
- Không dùng `setState()` cho mọi thứ

PhucPT10@fpt.edu.vn

IV. MỘT SỐ NỘI DUNG CHI TIẾT

PHẦN A. REACTIVE THINKING TRONG FLUTTER

(Trọng tâm tư duy – quan trọng nhất của Slot 5)

I. TƯ DUY UI TRUYỀN THỐNG vs REACTIVE UI

◈ UI điều khiển (Imperative / truyền thống)

- UI được điều khiển bằng:
  - biến
  - `setState()`
- Tư duy phổ biến của sinh viên:
  "Dữ liệu đổi → tôi phải nhớ gọi setState"

✕ *Vấn đề của UI truyền thống*

- Khi app lớn:
  - quá nhiều `setState`
  - khó biết **ai đang cập nhật UI**
- Dữ liệu đổi ở nhiều nơi → UI dễ **lệch trạng thái**
- Rất khó áp dụng cho:
  - chat
  - notification
  - realtime data

❗ *Sinh viên hay hiểu sai*

- Nghĩ rằng:
  "Flutter luôn cần setState"
- Thực tế:
  - `setState` chỉ là **một cách**, không phải **cách chuẩn cho app hiện đại**

◈ Reactive UI – UI phản ứng theo dữ liệu

- **Reactive = UI không điều khiển dữ liệu**
- UI **chỉ lắng nghe dữ liệu**
- Dữ liệu đổi → UI tự cập nhật

👉 Ví dụ:

- Counter realtime
- Tin nhắn chat mới
- Notification đến

👉 UI **không cần biết**:

- dữ liệu đổi khi nào
- đổi bằng cách nào

🎓 Ý nghĩa khi làm app thật

- App càng realtime → **càng không thể dùng tư duy cũ**
- Reactive UI giúp:
  - code rõ ràng
  - ít bug
  - dễ mở rộng

II. ASYNCHRONOUS TRONG DART – BẢN CHẤT

◈ Đồng bộ vs Bất đồng bộ (giải thích đơn giản)

- **Đồng bộ**:
  - Làm xong việc A → mới làm việc B
- **Bất đồng bộ**:
  - Giao việc A → tiếp tục làm việc khác
  - Khi A xong → nhận kết quả sau

PhucPT10@fpt.edu.vn

◈ Event Loop (mô tả nhẹ)

- Dart có một **vòng lặp sự kiện**
- Các tác vụ:
  - UI
  - input người dùng
  - async task

    đều được xếp hàng xử lý

👉 Nếu **block UI thread**:

- app giật
- app đứng
- trải nghiệm rất tệ

🔍 Nhấn mạnh quan trọng

- Flutter UI **KHÔNG ĐƯỢC block**
- Async **không phải tùy chọn**
- Mọi app thật đều:
  - gọi API
  - đọc dữ liệu
  - xử lý chậm

III. FUTURE – BẤT ĐỒNG BỘ MỘT LẦN

◈ Future là gì?

- Đại diện cho **một kết quả trong tương lai**
- Chỉ có **1 kết quả**
- Xong là **kết thúc**

◈ Khi nào Future là đủ?

- Load dữ liệu:
  - profile
  - danh sách tĩnh
- Gọi API:
  - login
  - submit form

◈ FutureBuilder hoạt động thế nào?

- Nhận Future
- Lắng nghe trạng thái:
  - loading
  - success
  - error
- Build UI **một lần theo kết quả**

✕ Giới hạn của Future

- Không phù hợp cho:
  - dữ liệu thay đổi liên tục
  - realtime
- Muốn cập nhật lại → phải tạo Future mới

❗ Lỗi sinh viên hay mắc

- Gọi Future trong `build()`
- Mỗi rebuild → Future chạy lại

PhucPT10@fpt.edu.vn

- UI load lại không kiểm soát

IV. STREAM – BẤT ĐỒNG BỘ LIÊN TỤC (CỐT LÕI SLOT 5)

◈ Stream là gì?

- **Luồng dữ liệu theo thời gian**
- Có thể phát:
  - 0
  - 1
  - nhiều giá trị

👉 Ví dụ:

- Tin nhắn chat
- Counter tăng liên tục
- Trạng thái đơn hàng

◈ Dữ liệu theo "dòng chảy"

- Dữ liệu **không đứng yên**
- Mỗi thay đổi → phát ra giá trị mới

◈ Stream lifecycle (mô tả đơn giản)

- Stream được tạo
- Phát dữ liệu
- Có thể:
  - tiếp tục
  - kết thúc

🔄 Stream vs Future (so sánh nhanh)

Tiêu chí	Future	Stream
Số lần trả dữ liệu	1	Nhiều
Realtime	❌	✅
UI auto-update	❌	✅

V. STREAMCONTROLLER – NGUỒN PHÁT DỮ LIỆU

◈ StreamController là gì?

- Công cụ:
  - tạo Stream
  - **chủ động phát dữ liệu**

◈ Các thao tác cơ bản

- `add(data)` → phát dữ liệu
- `close()` → đóng Stream

◈ Single vs Broadcast (giải thích nhẹ)

- **Single-subscription**:
  - 1 người nghe
- **Broadcast**:
  - nhiều UI cùng nghe

⚠ Những lỗi rất nguy hiểm

- ❌ Không `close()` controller → leak bộ nhớ
- ❌ Tạo StreamController trong UI

PhucPT10@fpt.edu.vn

- ❌ UI tự `add()` dữ liệu

👉 **UI không phải nguồn dữ liệu**

VI. STREAMBUILDER – UI PHẢN ỨNG THEO DỮ LIỆU

◈ StreamBuilder hoạt động thế nào?

- Lắng nghe Stream
- Mỗi khi Stream emit:
  - builder được gọi lại
  - UI rebuild

◈ snapshot gồm:

- `data`
- `error`
- `connectionState`

🔍 Vì sao StreamBuilder thay thế `setState`

- Không cần nhớ "khi nào gọi setState"
- UI **tự động** cập nhật khi dữ liệu đổi

✕ Sai lầm phổ biến

- Nhét logic xử lý vào `builder`
- Stream emit quá nhiều → rebuild loạn

PHẦN B. REPOSITORY PATTERN – KIẾN TRÚC NỀN TẢNG

VII. VÌ SAO CẦN REPOSITORY?

◈ Vấn đề nếu không có Repository

- UI gọi API trực tiếp
- Logic nằm trong Widget
- Code:
  - khó test
  - khó maintain
  - khó mở rộng

◈ Repository giải quyết gì?

- Tách:
  - UI
  - logic
  - data
- UI **không biết dữ liệu từ đâu**

VIII. REPOSITORY LÀ GÌ? (MỨC NỀN TẢNG)

- Repository là:
  - trung tâm dữ liệu
  - nơi tạo Stream
- Repository:
  - không phải UI
  - không phải Widget

🔄 Luồng chuẩn

`UI`

PhucPT10@fpt.edu.vn

```
 ↓
StreamBuilder
 ↓
Stream
 ↓
Repository
 ↓
Data Source (API / local / mock)
```

IX. KẾT HỢP STREAM + REPOSITORY

◈ Vì sao đây là "xương sống" Flutter hiện đại?

- Dữ liệu tập trung
- UI reactive
- Chuẩn bị sẵn cho:
  - BLoC
  - Riverpod
  - Clean Architecture

👉 Slot 5 **chưa dùng state management nâng cao**

👉 Nhưng **tư duy đã đúng chuẩn dự án thật**

X. LỖI TƯ DUY & HIỂU NHẦM NGHIÊM TRỌNG

❌ Lạm dụng setState

- Nguyên nhân: tư duy imperative
- Dấu hiệu: UI rối, bug ngầm
- Cách sửa: chuyển sang Reactive

❌ Dùng Stream cho mọi thứ

- Nguyên nhân: hiểu sai Stream = xịn hơn
- Cách sửa:
  - dữ liệu tĩnh → Future
  - dữ liệu động → Stream

❌ Không phân biệt UI – logic – data

- Nguyên nhân: học chưa theo kiến trúc
- Cách sửa:
  - UI chỉ hiển thị
  - Repository xử lý dữ liệu

XI. TỔNG KẾT & ĐỊNH HƯỚNG

- **Stream & Reactive là bước ngoặt tư duy**
- Slot 5 là:
  - cầu nối giữa Flutter cơ bản
  - và Flutter chuyên nghiệp
- Chuẩn bị trực tiếp cho:
  - State Management
  - App lớn
  - Team development

🎯 TRẢ LỜI CÂU HỎI

"Nếu làm app thật, kiến thức này giúp gì?"

👉 Giúp:

- UI luôn đúng với dữ liệu

PhucPT10@fpt.edu.vn

- Ít bug
- Dễ mở rộng
- Không sợ realtime

PhucPT10@fpt.edu.vn

V. KINH NGHIỆM THỰC TIỄN, TIPS

1 KỸ THUẬT: Future vs Stream (Async nền tảng)

✅ Kinh nghiệm thực tế trong dự án Flutter

- **Future** thường xuất hiện ở:
  - login
  - load profile
  - submit form
- **Stream** xuất hiện ở:
  - chat
  - cart
  - notification
  - dashboard realtime

👉 Trong dự án thật, **80% lỗi async** đến từ việc **dùng sai công cụ**.

🔑 Tình huống thực tế

- **Bối cảnh**: Màn hình danh sách đơn hàng
- **Vấn đề**: Trạng thái đơn thay đổi nhưng UI không cập nhật
- **Nguyên nhân**: Dùng `FutureBuilder` cho dữ liệu **thay đổi liên tục**
- **Vì sao chọn Stream**:
  - Không cần reload
  - UI auto-update

⚠️ Lỗi thường gặp

❌ **Lỗi 1: Dùng Stream cho mọi thứ**

- 🔍 Nguyên nhân:
  - Nghĩ Stream "xịn hơn"
- 🧪 Dấu hiệu:
  - Code phức tạp không cần thiết
- 🛠 Cách khắc phục:
  - Dữ liệu tĩnh → Future
  - Dữ liệu động → Stream
- 🚫 Phòng tránh:
  - Hỏi: *"Dữ liệu có đổi nhiều lần không?"*

💡 Tips & Tricks

- Đặt tên rõ:
  - `userProfileFuture`
  - `cartStream`
- Log **điểm phát dữ liệu**, không log trong UI

🛡 Best Practices

- ❌ Không "Stream hóa" mọi Future
- ✅ Chọn đúng công cụ cho đúng bài toán
- 🔄 Kết hợp:
  - Future cho init
  - Stream cho update

🎓 Bài học rút ra

Stream không thay thế Future – Stream giải quyết bài toán khác.

PhucPT10@fpt.edu.vn

2 KỸ THUẬT: StreamController

✅ Kinh nghiệm thực tế

- StreamController **không dùng trong UI**
- Thường nằm trong:
  - Repository
  - Service
- Là nơi **chủ động phát dữ liệu**

💡 Tình huống thực tế

- **Bối cảnh**: Cart trong app bán hàng
- **Vấn đề**: UI update sai số lượng
- **Nguyên nhân**: Mỗi màn hình tạo 1 StreamController
- **Giải pháp**: 1 controller – 1 nguồn dữ liệu

⚠️ Lỗi thường gặp

❌ Lỗi 1: Không `close()` StreamController

- 🔍 Nguyên nhân:
  - Không hiểu lifecycle
- 🧪 Dấu hiệu:
  - Memory tăng dần
- 🛠 Khắc phục:
  - Close trong `dispose()` của Repository
- 🚫 Phòng tránh:
  - Viết `dispose()` ngay khi tạo controller

❌ Lỗi 2: Tạo controller trong Widget

- 🔍 Nguyên nhân:
  - Nhầm UI = logic
- 🛠 Cách sửa:
  - Đưa controller ra Repository

💡 Tips & Tricks

- Dùng `debugPrint` khi `add()`
- Đặt tên:
  - `_cartController`
- Chỉ **1 nơi được add dữ liệu**

🛡 Best Practices

- UI **không add dữ liệu**
- Controller sống cùng lifecycle dữ liệu
- Broadcast **chỉ khi cần nhiều UI lắng nghe**

🎓 Bài học rút ra

StreamController là "nguồn phát", không phải đồ chơi trong UI.

3 KỸ THUẬT: StreamBuilder

✅ Kinh nghiệm thực tế

- StreamBuilder xuất hiện ở:
  - màn hình realtime
  - danh sách phụ thuộc dữ liệu
- Là **cầu nối duy nhất** giữa Stream & UI

PhucPT10@fpt.edu.vn

Tình huống thực tế

- **Bối cảnh**: Chat screen
- **Vấn đề**: UI rebuild liên tục, lag
- **Nguyên nhân**:
  - Stream emit quá nhiều
  - Logic nằm trong `builder`

Lỗi thường gặp

Lỗi 1: Nhét logic vào builder

- Nguyên nhân:
  - Nghĩ builder = controller
- Dấu hiệu:
  - Code dài
  - Debug khó
- Khắc phục:
  - Xử lý logic trước khi add vào Stream

Lỗi 2: Tạo Stream trong build()

- Nguyên nhân:
  - Không hiểu rebuild
- Khắc phục:
  - Tạo Stream ở Repository

Tips & Tricks

- Builder chỉ làm:
  - check snapshot
  - render UI
- Dùng widget con để giảm rebuild

Best Practices

- UI càng "ngu" càng tốt
- Không xử lý data trong builder
- StreamBuilder ≠ nơi xử lý logic

Bài học rút ra

StreamBuilder là "người nghe", không phải "người quyết định".

4 KỸ THUẬT: Repository Pattern (mức nền tảng)

Kinh nghiệm thực tế

- Repository luôn xuất hiện **trước khi app lớn**
- Là nơi:
  - tạo Stream
  - gọi API
  - xử lý dữ liệu

Tình huống thực tế

- **Bối cảnh**: App nội bộ nhiều màn hình
- **Vấn đề**: Mỗi màn hình fetch data riêng
- **Hậu quả**:
  - data không đồng bộ
- **Giải pháp**:

PhucPT10@fpt.edu.vn

    - Repository trung tâm

⚠ Lỗi thường gặp

❌ Lỗi 1: Repository cho có

- 🔍 Nguyên nhân:
    - Chưa thấy lợi ích
- 🛠 Cách sửa:
    - Mọi dữ liệu phải đi qua Repository

❌ Lỗi 2: UI biết quá nhiều

- 🔍 Nguyên nhân:
    - UI gọi API trực tiếp
- 🛠 Cách sửa:
    - UI chỉ gọi repository

💡 Tips & Tricks

- Mỗi loại dữ liệu → 1 repository
- Dễ mock khi test

🛡 Best Practices

- Repository là **nguồn chân lý**
- Chuẩn bị sẵn cho:
    - Bloc
    - Riverpod
- Không phụ thuộc UI

🎓 Bài học rút ra

Không có Repository → app lớn sẽ rối.

5 KỸ THUẬT: Reactive UI Pattern (Tổng hợp)

✅ Kinh nghiệm thực tế

- App càng realtime → càng cần reactive
- `setState` chỉ phù hợp demo nhỏ

⚠ Lỗi tư duy nguy hiểm

- Lạm dụng setState
- Không phân biệt:
    - UI
    - logic
    - data
- Dùng Stream "cho giống bài"

🛡 Nguyên tắc vàng

- **Dữ liệu là nguồn chân lý**
- UI chỉ phản ứng
- Repository là trung tâm

🎓 Nếu chỉ nhớ 2 ý

1. **UI không xử lý logic**
2. **Dữ liệu đổi → UI tự đổi**

PhucPT10@fpt.edu.vn

🎯 TỔNG KẾT CUỐI SLOT 5

Sau phần này, sinh viên phải hiểu rõ:

- ❌ Code ví dụ ≠ code dự án thật
- ✅ Vì sao app lớn **không dùng setState tràn lan**
- ✅ Những lỗi sẽ gặp **trước khi gặp thật**
- ✅ Hình thành:
  - tư duy Reactive
  - tư duy kiến trúc
  - tư duy làm app thật

👉 **Slot 5 kết thúc ở đây là hợp lý để bước sang Slot 6 – State Management nâng cao**, vì:

- tư duy đã sẵn
- lỗi cơ bản đã biết
- kiến trúc nền đã hình thành

PhucPT10@fpt.edu.vn

EXERCISE

BÀI TẬP TỔNG HỢP 1

LIVE COUNTER STREAM

(Realtime Data – Stream – StreamBuilder)

1 TỔNG QUAN DỰ ÁN

Tên dự án

Live Counter Stream

Mô tả ngắn gọn

Ứng dụng Flutter mô phỏng **dữ liệu thay đổi liên tục theo thời gian** (counter tăng dần mỗi giây).
UI **tự động cập nhật** khi có dữ liệu mới, **không dùng setState**.

Mục tiêu học tập

Sau dự án, sinh viên sẽ:

- Hiểu **Stream ≠ Future**
- Thấy rõ:
  - Vì sao UI có thể tự cập nhật liên tục
- Biết dùng **StreamBuilder đúng tư duy**
- Hiểu vai trò:
- `Stream → StreamBuilder → UI`

Đối tượng phù hợp

- Sinh viên vừa học xong **Slot 5**
- Đã biết:
  - StatelessWidget
  - async / await (cơ bản)
  - Future (Slot 4)

2 CHỨC NĂNG CHÍNH CỦA ỨNG DỤNG

Chức năng cốt lõi

- Counter **tăng tự động mỗi 1 giây**
- Dữ liệu phát ra liên tục bằng **Stream**
- UI:
  - hiển thị loading
  - hiển thị dữ liệu
  - hiển thị lỗi (mô phỏng)

Gắn với kiến thức Slot 5

Chức năng	Kỹ thuật
Counter realtime	Stream
Phát dữ liệu	StreamController
UI cập nhật	StreamBuilder
Không setState	Reactive UI

3 CÔNG NGHỆ & CÔNG CỤ

- Flutter SDK
- Dart
- VS Code / Android Studio
- Android Emulator
- Material Design

4 CẤU TRÚC THƯ MỤC DỰ ÁN (TẬP TRUNG STREAM)

```
lib/
```


PhucPT10@fpt.edu.vn

```
├── main.dart
├── services/
│   └── counter_stream_service.dart
├── screens/
│   └── home_screen.dart
```

🧠 Giải thích sư phạm

- ❌ UI **KHÔNG** tạo Stream
- ✅ Service chịu trách nhiệm:
  - tạo StreamController
  - phát dữ liệu
- UI **chỉ lắng nghe**

5 DANH SÁCH CÁC BƯỚC THỰC HIỆN (TÓM TẮT)

1. Tạo project Flutter
2. Chuẩn bị cấu trúc thư mục
3. Tạo CounterStreamService
4. Tạo StreamController
5. Phát dữ liệu theo thời gian
6. Expose Stream cho UI
7. Tạo HomeScreen
8. Kết nối Stream bằng StreamBuilder
9. Xử lý loading / data / error
10. Quản lý vòng đời (close Stream)
11. Chạy & quan sát app

6 HƯỚNG DẪN CHI TIẾT TỪNG BƯỚC (QUAN TRỌNG NHẤT)

🧩 BƯỚC 1. TẠO PROJECT

Mục tiêu:
Khởi tạo môi trường Flutter.

```
flutter create live_counter_stream
cd live_counter_stream
```

🧩 BƯỚC 2. TẠO SERVICE STREAM

📁 `lib/services/counter_stream_service.dart`

🎯 Mục tiêu

- Hiểu:
  - Stream không tự sinh dữ liệu
  - Phải có nơi **phát dữ liệu**

🛠 Code

```
import 'dart:async';

class CounterStreamService {
  final StreamController<int> _controller = StreamController<int>();

  Stream<int> get counterStream => _controller.stream;

  int _count = 0;
  Timer? _timer;

  void start() {
    _timer = Timer.periodic(Duration(seconds: 1), (timer) {
```


PhucPT10@fpt.edu.vn

```
      _count++;
      _controller.add(_count);
    });
  }

  void dispose() {
    _timer?.cancel();
    _controller.close();
  }
}
```

Kiểm tra tư duy

- Stream **chỉ là luồng**
- `add()` mới phát dữ liệu

⚠ Lỗi hay gặp:

- Quên `close()` → memory leak

BƯỚC 3. TẠO HOME SCREEN

`lib/screens/home_screen.dart`

Mục tiêu

- UI **không xử lý logic**
- UI **chỉ lắng nghe Stream**

Code

```
import 'package:flutter/material.dart';
import '../services/counter_stream_service.dart';

class HomeScreen extends StatelessWidget {
  final CounterStreamService _service = CounterStreamService();

  HomeScreen({super.key}) {
    _service.start();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Live Counter Stream')),
      body: Center(
        child: StreamBuilder<int>(
          stream: _service.counterStream,
          builder: (context, snapshot) {
            if (snapshot.connectionState == ConnectionState.waiting) {
              return const CircularProgressIndicator();
            }

            if (snapshot.hasError) {
              return const Text('Có lỗi xảy ra');
            }

            if (!snapshot.hasData) {
              return const Text('Chưa có dữ liệu');
            }

            return Text(
```


PhucPT10@fpt.edu.vn

```
                'Counter: ${snapshot.data}',
                style: const TextStyle(fontSize: 32),
              );
            },
          ),
        ),
      );
    }
}
```

Giải thích

- Không có `setState`
- Mỗi lần Stream phát → UI tự rebuild

BƯỚC 4. KẾT NỐI VỚI main.dart

`lib/main.dart`

```
import 'package:flutter/material.dart';
import 'screens/home_screen.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: HomeScreen(),
    );
  }
}
```

7 KIỂM TRA & CHẠY ỨNG DỤNG

```
flutter run
```

Quan sát

- Counter tăng mỗi giây
- UI cập nhật **không reload app**
- Không gọi `setState`

CÂU HỎI BẮT BUỘC CHO SINH VIÊN:

"Dữ liệu đến từ đâu?
Vì sao UI tự đổi?"

8 CÁC LỖI PHỔ BIẾN & CÁCH XỬ LÝ

Lỗi	Nguyên nhân	Cách sửa
UI không cập nhật	Không gọi `add()`	Kiểm tra StreamController
App chậm	Stream tạo trong build	Tạo Stream ngoài UI
Memory leak	Không close Stream	Gọi `dispose()`

9 KẾT QUẢ ĐẠT ĐƯỢC SAU DỰ ÁN

Sinh viên:

- ✅ Hiểu Stream là **bất đồng bộ nhiều lần**

PhucPT10@fpt.edu.vn

- ✅ Không còn nhầm:
  - Stream = Future chạy nhiều lần ❌
- ✅ Biết:
  - UI chỉ lắng nghe
  - Dữ liệu điều khiển UI
- ✅ Sẵn sàng học:
  - Reactive Pattern
  - State Management (Slot 6)

10 BÀI TẬP MỞ RỘNG (STREAM)

1. Thay đổi thời gian phát dữ liệu (0.5s / 2s)
2. Thêm nút **Pause / Resume**
3. So sánh:
   - setState counter
   - Stream counter
4. Log dữ liệu Stream ra console
5. Thêm lỗi giả lập (`addError()`)

PhucPT10@fpt.edu.vn

BÀI TẬP TỔNG HỢP 2

STREAM & UI REACTIVE – PRACTICE PROJECT

1 TỔNG QUAN DỰ ÁN

Tên dự án

Live Counter Stream App

Mô tả ngắn gọn

Ứng dụng Flutter mô phỏng **dữ liệu thay đổi liên tục theo thời gian** (counter realtime).
Dữ liệu được phát ra từ **Stream**, UI **tự động cập nhật** thông qua `StreamBuilder` mà **không cần gọi `setState()`**.

Mục tiêu học tập

Sau dự án, sinh viên **phải trả lời được bằng lời**:

- Stream là gì? Khác Future ở điểm nào?
- Vì sao UI có thể cập nhật nhiều lần?
- Vì sao UI **tự rebuild** mà không cần `setState`?
- UI Reactive là gì **(không phải khái niệm mơ hồ)**?

Đối tượng phù hợp

- Sinh viên **vừa hoàn thành Slot 5 – Stream**
- Trình độ **trung bình – yếu**
- Chưa học State Management nâng cao

2 CHỨC NĂNG CHÍNH CỦA ỨNG DỤNG

Chức năng cốt lõi

Chức năng	Mô tả
Counter realtime	Giá trị số tăng dần theo thời gian
UI tự cập nhật	Không cần reload, không setState
Trạng thái loading	Khi Stream chưa phát dữ liệu
Trạng thái error	Khi Stream phát lỗi (mô phỏng)

Gắn với Stream & StreamBuilder

- **Stream nào?**
  `Stream<int>` phát counter mỗi 1 giây
- **StreamBuilder nào?**
  `StreamBuilder<int>` trong `HomeScreen`
- **Snapshot xử lý ra sao?**
  - `waiting` → loading
  - `hasData` → hiển thị counter
  - `hasError` → hiển thị lỗi

3 CÔNG NGHỆ & CÔNG CỤ SỬ DỤNG

- Flutter SDK
- Dart
- Android Studio / VS Code
- Emulator
- Material Widgets

Không dùng:

- Provider / Bloc

PhucPT10@fpt.edu.vn

- RxDart
- Clean Architecture nâng cao

4 CẤU TRÚC THƯ MỤC DỰ ÁN (TẬP TRUNG STREAM)

```
lib/
├── main.dart
├── services/
│   └── stream_service.dart
├── screens/
│   └── home_screen.dart
```

📌 Giải thích bắt buộc cho sinh viên

- `StreamService`
  👉 **Nơi tạo & phát dữ liệu**
- `HomeScreen`
  👉 **Chỉ lắng nghe Stream & hiển thị UI**

🚫 UI **KHÔNG tạo Stream**
🚫 UI **KHÔNG xử lý logic dữ liệu**

5 DANH SÁCH CÁC BƯỚC THỰC HIỆN (TÓM TẮT)

1. Tạo Flutter project
2. Tạo `StreamService`
3. Khởi tạo `StreamController`
4. Phát dữ liệu theo thời gian
5. Kiểm tra Stream bằng `listen`
6. Kết nối Stream với UI bằng `StreamBuilder`
7. Quan sát UI rebuild
8. Xử lý snapshot đầy đủ
9. So sánh với `setState`
10. Ngăn logic trong UI
11. Đóng Stream
12. Hoàn thiện giao diện

6 HƯỚNG DẪN CHI TIẾT TỪNG BƯỚC (QUAN TRỌNG NHẤT)

◈ BƯỚC 1 – TẠO PROJECT FLUTTER

📌 *Mục tiêu*

Thiết lập môi trường để tập trung **quan sát Stream & UI**, không bị nhiễu kiến trúc.

🛠 *Thao tác*

```
flutter create live_counter_stream
cd live_counter_stream
flutter run
```

✅ *Kiểm chứng*

App chạy được trên emulator.

◈ BƯỚC 2 – TẠO STREAM SERVICE

📌 *Mục tiêu*

Hiểu rõ:

👉 **Stream phải được tạo ở nơi KHÔNG phải UI**

🛠 *Thao tác*

📂 Tạo file:

PhucPT10@fpt.edu.vn

`lib/services/stream_service.dart`

📋 Code minh họa:

```
import 'dart:async';

class StreamService {
  final StreamController<int> _controller = StreamController<int>();
  int _counter = 0;

  Stream<int> get stream => _controller.stream;

  void start() {
    Timer.periodic(Duration(seconds: 1), (timer) {
      _counter++;
      _controller.add(_counter);
    });
  }

  void dispose() {
    _controller.close();
  }
}
```

Tip

- `add()` = phát dữ liệu
- `stream` = UI sẽ lắng nghe

BƯỚC 3 – KIỂM TRA STREAM BẰNG LISTEN

Mục tiêu

Chứng minh: **Stream hoạt động độc lập với UI**

Thao tác

Trong `main.dart` (tạm thời):

```
final service = StreamService();
service.stream.listen((value) {
  print('Stream emits: $value');
});
service.start();
```

Kiểm chứng

Console in ra số tăng dần.

BƯỚC 4 – TẠO HOME SCREEN (UI)

Mục tiêu

UI **chỉ hiển thị**, không xử lý logic.

📂 Tạo file:

`lib/screens/home_screen.dart`

BƯỚC 5 – KẾT NỐI STREAM VỚI UI BẰNG STREAMBUILDER

Mục tiêu

Hiểu luồng:

Stream → StreamBuilder → UI

Code minh họa

```
StreamBuilder<int>(
  stream: streamService.stream,
  builder: (context, snapshot) {
```

```
    if (snapshot.connectionState == ConnectionState.waiting) {
      return CircularProgressIndicator();
    }

    if (snapshot.hasError) {
      return Text('Error: ${snapshot.error}');
    }

    if (!snapshot.hasData) {
      return Text('No data');
    }

    return Text(
      'Counter: ${snapshot.data}',
      style: TextStyle(fontSize: 32),
    );
  },
);
```

Sinh viên phải hiểu

- `builder` chạy **nhiều lần**
- UI đổi vì **snapshot đổi**

BƯỚC 6 – QUAN SÁT UI REBUILD

Mục tiêu

Chứng minh UI Reactive là **hành vi**, không phải lý thuyết.

Thêm log:

```
print('UI rebuild: ${snapshot.data}');
```

Kiểm chứng

- Log in liên tục
- UI đổi liên tục
- Không gọi setState

BƯỚC 7 – SO SÁNH VỚI SETSTATE (CHỈ ĐỂ NHẬN DIỆN)

Mục tiêu

Hiểu vì sao `setState` **không phù hợp cho realtime**

Cho sinh viên thử:

- Gắn logic vào UI
- Dùng `setState`

So sánh:

- Code rối
- UI gánh logic

BƯỚC 8 – ĐÓNG STREAM (NHẬN THỨC LIFECYCLE)

Mục tiêu

Nhận diện memory leak **ngay từ giai đoạn học cơ bản**.

```
void dispose() {
  _controller.close();
}
```

7 KIỂM TRA & CHẠY ỨNG DỤNG

```
flutter run
```


PhucPT10@fpt.edu.vn

👉 Quan sát:

- UI đổi liên tục
- Không reload
- Không setState

8 CÁC LỖI PHỔ BIẾN & CÁCH XỬ LÝ

Lỗi	Nguyên nhân
UI không rebuild	Không dùng StreamBuilder
Snapshot null	Không xử lý waiting
App chậm dần	Quên close Stream
Nhầm FutureBuilder	Hiểu sai Stream

9 KẾT QUẢ ĐẠT ĐƯỢC SAU DỰ ÁN

Sinh viên **PHẢI**:

- Hiểu bản chất Stream
- Biết khi nào dùng Stream
- Hiểu UI Reactive là gì
- Sẵn sàng học State Management

10 BÀI TẬP MỞ RỘNG (STREAM-CENTERED)

- Thay đổi tốc độ counter
- Pause / resume Stream
- Log dữ liệu Stream
- So sánh Stream vs setState bằng báo cáo

🎯 THÔNG ĐIỆP CUỐI CÙNG

Dữ liệu thay đổi → UI phản ứng
UI không điều khiển dữ liệu
Stream là nền tảng của Reactive Thinking

PhucPT10@fpt.edu.vn

BÀI TẬP TỔNG HỢP 3

REPOSITORY PATTERN – PRACTICE PROJECT

(Foundation Level – Không framework – Đúng tư duy)

1 TỔNG QUAN DỰ ÁN

Tên dự án

User List Repository Demo

Mô tả ngắn gọn

Ứng dụng Flutter hiển thị **danh sách người dùng (User List)**.
Điểm quan trọng **KHÔNG nằm ở UI**, mà nằm ở cách **UI lấy dữ liệu**:

- UI **không gọi fake data**
- UI **không gọi API**
- UI **chỉ gọi Repository**

Repository chịu trách nhiệm **ẩn toàn bộ nguồn dữ liệu**.

Mục tiêu học tập (CỰC KỲ QUAN TRỌNG)

Sau dự án, sinh viên **PHẢI GIẢI THÍCH ĐƯỢC BẰNG LỜI**:

- Repository Pattern là gì (bản chất, không phải định nghĩa sách)
- Vì sao UI **không nên** gọi API / fake data / Stream trực tiếp
- Repository **khác gì Service**
- Repository giúp ích gì khi:
  - đổi nguồn dữ liệu
  - mở rộng app
  - làm việc nhóm

Đối tượng phù hợp

- Sinh viên vừa học xong **Slot 5 – Stream / Async**
- Đã biết:
  - `Future`
  - `async / await`
  - `StatelessWidget`
- **CHƯA** học Clean Architecture / State Management

2 BÀI TOÁN THỰC TẾ DẪN NHẬP (BẮT BUỘC)

KỊCH BẢN BAN ĐẦU (CỐ TÌNH LÀM SAI)

Giả sử sinh viên viết code như sau:

```
// Trong UI
final users = [
  User(id: 1, name: 'Alice'),
  User(id: 2, name: 'Bob'),
];
```

Hoặc:

```
// Trong UI
final response = await http.get(...);
```

Hoặc:

```
// Trong UI
fakeUserService.getUsers();
```

VẤN ĐỀ PHÁT SINH

Vấn đề	Hậu quả
UI biết dữ liệu từ đâu	Phụ thuộc chặt

PhucPT10@fpt.edu.vn

Đổi fake → API	Phải sửa UI
Test UI	Rất khó
Mở rộng app	Code rối
Làm việc nhóm	Dễ conflict

🎯 MỤC TIÊU GIẢI QUYẾT

"UI KHÔNG BIẾT DỮ LIỆU ĐẾN TỪ ĐÂU"

👉 Đây chính là **lý do Repository Pattern ra đời**

3 KIẾN TRÚC MỤC TIÊU (PHIÊN BẢN ĐƠN GIẢN)

```
UI (Screen)
   ↓
Repository (Interface)
   ↓
Data Source (Fake / API / Local)
```

🧠 Giải thích sư phạm

- UI **chỉ biết Repository**
- Repository **quyết định lấy dữ liệu từ đâu**
- Data Source có thể:
  - fake
  - API
  - local
- UI **KHÔNG BỊ ẢNH HƯỞNG** khi đổi Data Source

4 CẤU TRÚC THƯ MỤC DỰ ÁN (REPOSITORY-CENTERED)

```
lib/
├── main.dart
├── models/
│   └── user.dart
├── data/
│   ├── repositories/
│   │   └── user_repository.dart
│   ├── datasources/
│   │   └── user_fake_datasource.dart
├── screens/
│   └── user_screen.dart
```

📌 Giải thích BẮT BUỘC cho sinh viên

- `user_repository.dart`
  👉 **Hợp đồng (contract)** – UI làm việc với file này
- `user_fake_datasource.dart`
  👉 **Chi tiết kỹ thuật** – UI KHÔNG được import

⛔ UI **KHÔNG IMPORT datasource**

5 DANH SÁCH CÁC BƯỚC THỰC HIỆN (TÓM TẮT)

1. Tạo model `User`
2. Tạo fake data source
3. Tạo Repository interface (abstract class)
4. Implement Repository
5. UI gọi Repository
6. Quan sát UI KHÔNG phụ thuộc data source
7. Mô phỏng đổi data source
8. Tổng kết tư duy Repository

PhucPT10@fpt.edu.vn

6 HƯỚNG DẪN CHI TIẾT TỪNG BƯỚC (QUAN TRỌNG NHẤT)

◈ BƯỚC 1 – TẠO MODEL USER

📌 *Mục tiêu nhận thức*

Model giúp tách **dữ liệu** khỏi UI & Repository.

🛠 *Thao tác*

📂 Tạo file:

`lib/models/user.dart`

📋 Code minh họa:

```
class User {
  final int id;
  final String name;

  User({required this.id, required this.name});
}
```

◈ BƯỚC 2 – TẠO FAKE DATA SOURCE

📌 *Mục tiêu nhận thức*

Data Source = nơi **thực sự lấy dữ liệu**, KHÔNG phải nơi UI gọi.

🛠 *Thao tác*

📂 Tạo file:

`lib/data/datasources/user_fake_datasource.dart`

📋 Code minh họa:

```
import '../../models/user.dart';

class UserFakeDataSource {
  Future<List<User>> getUsers() async {
    await Future.delayed(Duration(seconds: 1));
    return [
      User(id: 1, name: 'Alice'),
      User(id: 2, name: 'Bob'),
    ];
  }
}
```

◈ BƯỚC 3 – TẠO REPOSITORY INTERFACE (CỐT LÕI)

📌 *Mục tiêu nhận thức (QUAN TRỌNG NHẤT)*

👉 Repository là **hợp đồng**, không phải implementation.

🛠 *Thao tác*

📂 Tạo file:

`lib/data/repositories/user_repository.dart`

📋 Code minh họa:

```
import '../../models/user.dart';

abstract class UserRepository {
  Future<List<User>> getUsers();
}
```

💡 Giảng cho sinh viên:

- UI chỉ biết **interface**
- UI không biết implementation

PhucPT10@fpt.edu.vn

◈ BƯỚC 4 – IMPLEMENT REPOSITORY

📌 Mục tiêu nhận thức

Repository **đứng giữa UI & Data Source**

🛠 Thao tác

📋 Code minh họa:

```
import 'user_repository.dart';
import '../datasources/user_fake_datasource.dart';
import '../../models/user.dart';

class UserRepositoryImpl implements UserRepository {
  final UserFakeDataSource dataSource;

  UserRepositoryImpl(this.dataSource);

  @override
  Future<List<User>> getUsers() {
    return dataSource.getUsers();
  }
}
```

📌 Nhấn mạnh:

- Repository **KHÔNG chứa UI logic**
- Repository **chỉ điều phối dữ liệu**

◈ BƯỚC 5 – UI GỌI REPOSITORY (KHÔNG GỌI DATASOURCE)

📌 Mục tiêu nhận thức

👉 UI **phụ thuộc Repository, không phụ thuộc dữ liệu**

🛠 Thao tác

📂 Mở file:

`lib/screens/user_screen.dart`

📋 Code minh họa:

```
class UserScreen extends StatelessWidget {
  final UserRepository repository;

  UserScreen(this.repository);

  @override
  Widget build(BuildContext context) {
    return FutureBuilder(
      future: repository.getUsers(),
      builder: (context, snapshot) {
        if (!snapshot.hasData) {
          return CircularProgressIndicator();
        }

        final users = snapshot.data as List<User>;

        return ListView(
          children: users
              .map((u) => ListTile(title: Text(u.name)))
              .toList(),
        );
```


PhucPT10@fpt.edu.vn

```
        },
      );
    }
}
```

◈ BƯỚC 6 – QUAN SÁT ĐIỀU QUAN TRỌNG

📌 Đặt câu hỏi cho sinh viên:

- UI có biết fake data không? ❌
- UI có biết API không? ❌
- UI biết gì? ✅ **Repository**

7 SO SÁNH TRƯỚC & SAU KHI DÙNG REPOSITORY

Tiêu chí	Không Repository	Có Repository
UI phụ thuộc dữ liệu	❌	✅
Đổi data source	❌	✅
Dễ test	❌	✅
Dự án lớn	❌	✅

8 CÁC LỖI PHỔ BIẾN & CÁCH XỬ LÝ

Lỗi	Nhận diện
UI gọi datasource	Sai tư duy
Repository chứa UI	Lẫn trách nhiệm
Không dùng abstract	Mất lợi ích
Hard-code trong UI	Phá kiến trúc

9 KẾT QUẢ ĐẠT ĐƯỢC SAU DỰ ÁN

Sinh viên **PHẢI**:

- Vẽ được sơ đồ Repository Pattern
- Giải thích được:
  - Vì sao cần Repository
  - Khi nào nên / không nên dùng
- Sẵn sàng học:
  - Clean Architecture
  - State Management

10 BÀI TẬP MỞ RỘNG (REPOSITORY-FOCUSED)

- Thay fake datasource bằng API (mô phỏng)
- Thêm local datasource
- Thêm cache logic trong Repository
- Tạo thêm `ProductRepository`

🎯 THÔNG ĐIỆP CUỐI

Repository không phải để "cho đẹp code"
Repository tồn tại để UI không phụ thuộc dữ liệu
Đây là nền móng của mọi kiến trúc Flutter chuyên nghiệp

PhucPT10@fpt.edu.vn

BÀI TẬP TỔNG HỢP 4

REACTIVE THINKING – PRACTICE PROJECT

(Foundation Level – Không framework – Đúng tư duy hệ thống)

1 TỔNG QUAN DỰ ÁN

Tên dự án

Reactive Counter Demo – From Imperative to Reactive UI

Mô tả ngắn gọn

Ứng dụng mô phỏng **một giá trị Counter thay đổi liên tục theo thời gian**.
Điểm trọng tâm **KHÔNG phải là Counter**, mà là:

- Ai giữ dữ liệu?
- Ai quyết định khi UI cập nhật?
- UI có "điều khiển" hay chỉ "phản ứng"?

UI sẽ **tự động thay đổi** khi dữ liệu đổi, **không gọi setState**.

Mục tiêu học tập

Sau dự án, sinh viên **PHẢI GIẢI THÍCH ĐƯỢC**:

- Reactive Thinking là gì (bằng lời, không phải định nghĩa sách)
- Khác biệt giữa:
  - **Imperative UI** (UI điều khiển)
  - **Reactive UI** (UI phản ứng)
- Khi dữ liệu đổi:
  - Ai chủ động?
  - UI có làm gì không?

Đối tượng phù hợp

- Sinh viên đã học:
  - Stream
  - StreamBuilder
  - Repository Pattern (mức nền tảng)
- Chưa học State Management nâng cao

2 BÀI TOÁN THỰC TẾ DẪN NHẬP (BẮT BUỘC)

Kịch bản quen thuộc

App có dữ liệu **thay đổi liên tục**:

- Counter tăng theo thời gian
- Trạng thái server (online / offline)
- Số message mới

✕ CÁCH LÀM SAI PHỔ BIẾN (IMPERATIVE)

```
int counter = 0;

void increase() {
  counter++;
  setState(() {});
}
```

Vấn đề tư duy

- UI:
  - giữ dữ liệu
  - xử lý logic
  - quyết định khi nào update

PhucPT10@fpt.edu.vn

- Dữ liệu & UI **dính chặt**

👉 **Đây KHÔNG phải Reactive**

🎯 Mục tiêu chuyển đổi tư duy

UI không điều khiển dữ liệu
UI chỉ phản ứng khi dữ liệu thay đổi

3 TƯ DUY REACTIVE – MÔ HÌNH ĐÚNG

```
DATA / LOGIC
      ↓
  STREAM
      ↓
REACTIVE WIDGET
      ↓
      UI
```

🧠 Giải thích cốt lõi

Câu hỏi	Trả lời
Ai phát dữ liệu?	Logic / Service
Ai lắng nghe?	UI
UI có giữ data không?	❌
UI có xử lý logic không?	❌
UI làm gì?	Render theo dữ liệu

👉 **UI là màn hình hiển thị, không phải bộ não**

4 CẤU TRÚC THƯ MỤC DỰ ÁN (REACTIVE-FOCUSED)

```
lib/
├── main.dart
├── services/
│   └── counter_stream_service.dart
├── screens/
│   └── home_screen.dart
```

📌 Giải thích BẮT BUỘC

- `counter_stream_service.dart`
  👉 **Nơi giữ & thay đổi dữ liệu**
- `home_screen.dart`
  👉 **Chỉ lắng nghe & hiển thị**

⛔ UI **KHÔNG tạo Stream**
⛔ UI **KHÔNG add dữ liệu**

5 DANH SÁCH CÁC BƯỚC THỰC HIỆN (TÓM TẮT)

1. Tạo Stream phát dữ liệu theo thời gian
2. Quan sát dữ liệu trong console
3. Kết nối Stream với UI
4. Quan sát UI tự cập nhật
5. So sánh với setState
6. Phân tích snapshot
7. Nhận diện UI reactive thật
8. Tổng kết tư duy

6 HƯỚNG DẪN CHI TIẾT TỪNG BƯỚC (CỐT LÕI)

PhucPT10@fpt.edu.vn

◈ BƯỚC 1 – TẠO STREAM PHÁT DỮ LIỆU

📌 Mục tiêu nhận thức

👉 **Dữ liệu tự thay đổi, không cần UI**

🛠 Thao tác

📂 Tạo file:
`lib/services/counter_stream_service.dart`

📋 Code minh họa:

```
import 'dart:async';

class CounterStreamService {
  final _controller = StreamController<int>();
  int _counter = 0;

  Stream<int> get counterStream => _controller.stream;

  void start() {
    Timer.periodic(Duration(seconds: 1), (_) {
      _counter++;
      print('Emit counter: $_counter');
      _controller.add(_counter);
    });
  }

  void dispose() {
    _controller.close();
  }
}
```

✅ Kiểm chứng

- Console in ra:
- `Emit counter: 1`
- `Emit counter: 2`
- `Emit counter: 3`

👉 **Chưa có UI vẫn chạy**

◈ BƯỚC 2 – KẾT NỐI STREAM VỚI UI

📌 Mục tiêu nhận thức

👉 UI **không gọi logic**, chỉ **nghe dữ liệu**

🛠 Thao tác

📂 Mở file:
`lib/screens/home_screen.dart`

📋 Code minh họa:

```
class HomeScreen extends StatelessWidget {
  final CounterStreamService service;

  HomeScreen(this.service);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
```


PhucPT10@fpt.edu.vn

```
      appBar: AppBar(title: Text('Reactive Counter')),
      body: Center(
        child: StreamBuilder<int>(
          stream: service.counterStream,
          builder: (context, snapshot) {
            print('UI rebuild');
            if (!snapshot.hasData) {
              return Text('Waiting...');
            }
            return Text(
              snapshot.data.toString(),
              style: TextStyle(fontSize: 40),
            );
          },
        ),
      ),
    );
  }
}
```

Kiểm chứng

- UI tự tăng số
- Không có `setState`
- Console in:
- `UI rebuild`
- `UI rebuild`

UI rebuild là phản ứng, không phải lỗi

BƯỚC 3 – SO SÁNH VỚI setState (NHẬN DIỆN TƯ DUY)

Mục tiêu nhận thức

Khác biệt **TƯ DUY**, không phải cú pháp

Imperative (sai tư duy)

- UI:
  - giữ biến
  - xử lý logic
  - gọi setState

Reactive (đúng tư duy)

- UI:
  - không giữ dữ liệu
  - không xử lý logic
  - chỉ render

7 BÀI THỰC HÀNH SO SÁNH (BẮT BUỘC)

Tiêu chí	Imperative UI	Reactive UI
UI giữ data	✅	❌
UI xử lý logic	✅	❌
Dữ liệu đổi	UI quyết	Logic quyết
Realtime	❌	✅
Mở rộng	❌	✅

PhucPT10@fpt.edu.vn

8 CÁC LỖI PHỔ BIẾN TRONG REACTIVE THINKING

❌ Lỗi 1: Xử lý logic trong builder

```
builder: (_, snapshot) {
  counter++;
}
```

👉 **Sai bản chất reactive**

❌ Lỗi 2: Dùng Stream nhưng vẫn setState

👉 **Reactive giả**

❌ Lỗi 3: Sợ UI rebuild

👉 Rebuild ≠ performance issue

👉 Sai Stream mới gây issue

9 KẾT QUẢ ĐẠT ĐƯỢC SAU DỰ ÁN

Sinh viên **PHẢI**:

- Giải thích được:
  - Reactive Thinking là gì
  - Flutter reactive ở điểm nào
- Nhận diện được:
  - UI reactive thật
  - UI reactive giả
- Sẵn sàng học:
  - State Management
  - Kiến trúc hướng dữ liệu

10 BÀI TẬP MỞ RỘNG (REACTIVE-FOCUSED)

- Thêm **2 Stream** cùng lúc
- 2 widget cùng lắng nghe 1 Stream
- Pause / Resume Stream
- Vẽ sơ đồ luồng dữ liệu

🎓 THÔNG ĐIỆP CUỐI CÙNG

Reactive không phải là Stream
Reactive là cách TƯ DUY
UI không điều khiển – UI phản ứng

PhucPT10@fpt.edu.vn

TỪ ĐIỂN THUẬT NGỮ (FOUNDATION GLOSSARY)

Thuật ngữ	Diễn giải đơn giản cho sinh viên	Ý nghĩa trong bài học	Ví dụ / Ghi chú sư phạm
Reactive Thinking	Cách tư duy trong đó **UI không điều khiển dữ liệu**, mà **chỉ phản ứng khi dữ liệu thay đổi**	Trọng tâm của toàn bộ dự án	"Dữ liệu đổi → UI tự đổi"
Imperative UI	UI **ra lệnh**: giữ dữ liệu, xử lý logic, gọi `setState`	Cách làm cũ, dễ lỗi	`count++; setState()`
Reactive UI	UI **lắng nghe dữ liệu**, không giữ state	Mục tiêu sinh viên cần đạt	`StreamBuilder`
Data-driven UI	UI được quyết định bởi **dữ liệu**, không phải hành vi người viết code	Bản chất của Flutter	Flutter = declarative
Stream	Dòng dữ liệu phát ra **theo thời gian**, nhiều lần	Trung tâm của reactive	Ví dụ: realtime status
Future	Dữ liệu trả về **1 lần trong tương lai**	So sánh với Stream	API call 1 lần
StreamController	Nơi **phát dữ liệu** vào Stream	Logic nằm ở đây, không phải UI	`controller.add(data)`
emit data	Hành động **đẩy dữ liệu mới** vào Stream	Làm UI thay đổi	Gọi `add()`
listen	Hành động **lắng nghe dữ liệu** từ Stream	UI làm việc này	`StreamBuilder`
build()	Hàm Flutter dùng để **vẽ UI**	Được gọi lại khi data đổi	Rebuild ≠ lỗi
UI rebuild	UI được vẽ lại khi có dữ liệu mới	Điều BẮT BUỘC trong reactive	Không phải performance issue
StreamBuilder	Widget giúp UI **lắng nghe Stream và tự cập nhật**	Công cụ chính của bài học	Không xử lý logic
FutureBuilder	Widget giúp UI hiển thị dữ liệu từ Future	Dùng để so sánh với Stream	Chạy 1 lần
snapshot	Dữ liệu hiện tại mà Stream/Future gửi cho UI	UI đọc snapshot để vẽ	`snapshot.data`
snapshot.hasData	Kiểm tra Stream đã có dữ liệu chưa	Tránh lỗi null	BẮT BUỘC kiểm tra
snapshot.connectionState	Trạng thái kết nối dữ liệu	Hiểu vòng đời async	loading / active
StatelessWidget	Widget **không giữ state nội bộ**	Phù hợp reactive	UI chỉ hiển thị

PhucPT10@fpt.edu.vn

setState	Cách ép Flutter vẽ lại UI thủ công	KHÔNG dùng làm trung tâm	Dễ sai tư duy
UI giữ state	UI lưu biến dữ liệu bên trong	Điều cần loại bỏ	`int count = 0;`
Business Logic	Phần xử lý dữ liệu, tính toán	KHÔNG đặt trong UI	Đặt ở Service
Service / Logic Layer	Nơi tạo & phát dữ liệu	Chủ động phát data	Không biết UI
Separation of Concerns	Tách rõ vai trò từng phần	Tư duy kiến trúc nền tảng	UI ≠ Logic
Snapshot null	Stream chưa phát dữ liệu	Trạng thái bình thường	Không phải lỗi
Realtime	Dữ liệu thay đổi liên tục	Lợi thế của Stream	Chat, status
Performance issue (hiểu nhầm)	Nghĩ rằng rebuild là chậm	Sai trong Flutter	Flutter tối ưu sẵn
Fake reactive UI	Dùng Stream nhưng vẫn `setState`	Sai tư duy	Rất hay gặp
True reactive UI	UI chỉ build theo snapshot	Mục tiêu cuối	Không điều khiển
Data Source	Nơi sinh ra dữ liệu ban đầu	Có thể là timer, API	Trong service
Declarative UI	Mô tả UI theo trạng thái	Flutter sử dụng cách này	Khác imperative
Observer pattern (ngầm hiểu)	Mô hình "nghe – phản ứng"	Nền tảng reactive	Stream áp dụng

NHÓM 1. TƯ DUY & KHÁI NIỆM NỀN TẢNG

1 Reactive UI (UI phản ứng theo dữ liệu)

Định nghĩa:
UI **không tự quyết định khi nào cập nhật**, mà **tự động thay đổi khi dữ liệu thay đổi**.
Hiểu đơn giản:
"Dữ liệu đổi → UI tự đổi theo"
Ý nghĩa thực tế:

- Phù hợp với app:
  - realtime
  - nhiều trạng thái
  - dữ liệu thay đổi liên tục

Sai lầm hay gặp:

- Nghĩ Reactive = phức tạp
- Thực chất: Reactive giúp **giảm setState và giảm bug**

2 UI điều khiển (Imperative UI)

Định nghĩa:
UI **chủ động ra lệnh cập nhật** bằng `setState()`.
Ví dụ tư duy:
"Dữ liệu đổi → gọi setState → UI vẽ lại"
Hạn chế trong app lớn:

- Khó kiểm soát
- Logic rải rác trong UI

PhucPT10@fpt.edu.vn

- Dễ lỗi khi nhiều trạng thái

3 Dữ liệu là nguồn chân lý (Single Source of Truth)

Định nghĩa:
Mọi trạng thái UI phải **xuất phát từ dữ liệu**, không từ biến tạm trong Widget.
Ý nghĩa:

- Tránh lệch trạng thái
- Dễ debug
- Dễ mở rộng

NHÓM 2. ASYNCHRONOUS TRONG DART

4 Synchronous (Đồng bộ)

Định nghĩa:
Code chạy **tuần tự**, chờ xong bước trước mới chạy bước sau.
Vấn đề với Flutter UI:

- Gây block UI
- App bị đơ

5 Asynchronous (Bất đồng bộ)

Định nghĩa:
Tác vụ chạy **song song**, không chặn UI.
Ý nghĩa trong Flutter:

- UI luôn mượt
- Bắt buộc với:
  - API
  - database
  - file
  - realtime

6 Event Loop

Định nghĩa (đơn giản):
Cơ chế Dart dùng để:

- xử lý async task
- cập nhật UI đúng thời điểm

Sinh viên cần nhớ:
Flutter chỉ có **1 UI thread** → không được block

NHÓM 3. FUTURE & FUTUREBUILDER

7 Future

Định nghĩa:
Đại diện cho **một kết quả sẽ có trong tương lai – chỉ 1 lần**.
Phù hợp khi:

- Load dữ liệu 1 lần
- Không cần realtime

Không phù hợp khi:

- dữ liệu thay đổi liên tục
- cần auto-update UI

8 FutureBuilder

Định nghĩa:
Widget xây dựng UI dựa trên **trạng thái của Future**.
Giới hạn:

PhucPT10@fpt.edu.vn

- Mỗi lần rebuild → Future có thể chạy lại
- Không phù hợp cho realtime

Sai lầm phổ biến:

- Gọi Future trong `build()`

🌊 NHÓM 4. STREAM – TRỌNG TÂM SLOT 5

9 Stream

Định nghĩa:
Luồng dữ liệu bất đồng bộ **phát ra nhiều giá trị theo thời gian**.
Hiểu hình ảnh:
Dòng nước → mỗi lần có nước mới → UI nhận
Phù hợp với:

- chat
- notification
- cart
- realtime dashboard

10 Stream lifecycle

Gồm các giai đoạn:

- create
- listen
- emit data
- close

Ý nghĩa:

- Phải quản lý vòng đời
- Không close → memory leak

11 StreamController

Định nghĩa:
Công cụ **tạo & điều khiển Stream**.
Vai trò:

- `add()` → phát dữ liệu
- `close()` → kết thúc Stream

Nguyên tắc:

- ❌ UI không tạo StreamController
- ✅ Repository / logic layer tạo

12 Single-subscription Stream

Định nghĩa:
Chỉ **1 listener** được subscribe.
Dùng khi:

- dữ liệu gắn với 1 màn hình

13 Broadcast Stream

Định nghĩa:
Nhiều listener cùng subscribe.
Dùng khi:

- nhiều màn hình
- global data (user, notification)

PhucPT10@fpt.edu.vn

🧩 NHÓM 5. STREAMBUILDER & UI

1 4 StreamBuilder

Định nghĩa:
Widget xây dựng UI **dựa trên dữ liệu phát ra từ Stream**.
Tư duy đúng:
UI không xử lý logic – UI chỉ lắng nghe

1 5 snapshot (AsyncSnapshot)

Định nghĩa:
Đối tượng chứa trạng thái Stream tại thời điểm build.
Gồm:

- `data`
- `error`
- `connectionState`

Sai lầm phổ biến:

- xử lý logic phức tạp trong builder

1 6 UI rebuild

Định nghĩa:
Flutter vẽ lại Widget khi dữ liệu thay đổi.
Quan trọng:

- Rebuild ≠ render lại toàn app
- Nhưng rebuild quá nhiều → tốn tài nguyên

🏗️ NHÓM 6. REPOSITORY PATTERN (NỀN TẢNG)

1 7 Repository Pattern

Định nghĩa:
Lớp **trung tâm quản lý dữ liệu**, tách khỏi UI.
Vai trò:

- gọi API
- xử lý data
- phát Stream

1 8 Data Source

Định nghĩa:
Nguồn dữ liệu thật:

- API
- database
- local storage

UI không nên biết trực tiếp

1 9 Luồng dữ liệu chuẩn (Data Flow)

```
Data Source
    ↓
Repository
    ↓
Stream
    ↓
StreamBuilder
    ↓
UI
```

Ý nghĩa:

PhucPT10@fpt.edu.vn

- UI thuần hiển thị
- Logic tập trung
- Dễ test & mở rộng

⚠ NHÓM 7. LỖI TƯ DUY & CẢNH BÁO

2 0 Logic trong UI

Định nghĩa:
Xử lý dữ liệu, gọi API, tạo Stream trong Widget.
Hậu quả:
- khó test
- khó maintain
- bug dây chuyền

2 1 Memory Leak

Định nghĩa:
Tài nguyên không được giải phóng dù không dùng nữa.
Nguyên nhân phổ biến:
- StreamController không `close()`

2 2 Over-engineering

Định nghĩa:
Dùng Stream / Reactive khi không cần thiết.
Dấu hiệu:
- code phức tạp
- không mang lại lợi ích rõ ràng

🎓 TỔNG KẾT CHO SINH VIÊN

Nếu chỉ nhớ **5 thuật ngữ cốt lõi của Slot 5**, hãy nhớ:
1. Reactive UI
2. Stream ≠ Future
3. StreamBuilder = UI lắng nghe
4. Repository = trung tâm dữ liệu
5. Dữ liệu là nguồn chân lý

PhucPT10@fpt.edu.vn

10 PROMPT TỰ NGHIÊN CỨU CHUYÊN SÂU

1 Stream vs Future dưới góc nhìn kiến trúc ứng dụng Flutter

So sánh Stream và Future không chỉ về cú pháp mà về tác động kiến trúc.
Trong những trường hợp nào Stream giúp ứng dụng **dễ mở rộng, dễ thay đổi yêu cầu** hơn Future?
Ngược lại, khi nào việc dùng Stream khiến kiến trúc trở nên **phức tạp, khó hiểu và khó bảo trì**?

👉 Gợi ý phân tích:
- Mối quan hệ giữa loại dữ liệu (1 lần vs liên tục)
- Chi phí tư duy & quản lý lifecycle
- Ảnh hưởng đến Repository & UI

2 Vòng đời StreamBuilder và hệ quả khi hiểu sai cơ chế rebuild

Phân tích vòng đời của StreamBuilder khi Widget rebuild nhiều lần.
Nếu sinh viên hiểu sai rằng "StreamBuilder chỉ rebuild khi có data mới", điều gì sẽ xảy ra trong app thật?

👉 Bắt buộc liên hệ:
- Performance
- Bug logic "khó tái hiện"
- Rebuild không kiểm soát

3 Repository Pattern và việc tách UI khỏi logic dữ liệu

Đánh giá vai trò thực sự của Repository Pattern trong Flutter.
Nếu UI gọi API trực tiếp hoặc xử lý dữ liệu ngay trong Widget, những vấn đề nào sẽ xuất hiện khi:
- thêm màn hình mới?
- đổi API?
- viết test?

👉 Không hỏi "Repository là gì", mà hỏi **"Nếu không có Repository thì hậu quả gì?"**

4 setState vs Reactive UI dựa trên Stream

Phân tích sự khác biệt về tư duy giữa setState và Reactive UI.
Vì sao setState phù hợp cho bài học nhập môn nhưng lại trở thành "điểm nghẽn kiến trúc" trong app lớn?

👉 Sinh viên phải lập luận:
- ai kiểm soát UI update?
- UI "chủ động" hay "phản ứng"?

5 Rủi ro khi lạm dụng Stream cho mọi loại dữ liệu

Đánh giá mặt trái của việc dùng Stream cho tất cả dữ liệu trong app.
Những dấu hiệu nào cho thấy bạn đang:
- dùng Stream chỉ vì "nghe có vẻ chuyên nghiệp"?
- biến app thành một hệ thống khó debug?

👉 Liên hệ:
- Over-engineering
- Cognitive load cho team

6 StreamController và thiết kế luồng dữ liệu đúng vai trò

Phân tích vai trò của StreamController trong kiến trúc Flutter.
Sự khác biệt giữa single-subscription và broadcast Stream ảnh hưởng thế nào đến:
- thiết kế UI?
- khả năng mở rộng nhiều màn hình dùng chung dữ liệu?

👉 Bắt buộc trả lời:
- Ai NÊN tạo StreamController?

- Ai KHÔNG NÊN?

7 Test & Debug: Stream + Repository vs Logic trong Widget

Đánh giá khả năng test và debug của hai cách tiếp cận:
(1) Logic nằm trong Widget
(2) Logic nằm trong Repository + Stream

👉 Sinh viên cần phân tích:

- test không cần UI
- debug theo luồng dữ liệu
- phát hiện bug sớm hay muộn

8 Reactive Pattern và khả năng maintain code trong team

Phân tích tác động của Reactive Pattern đến việc bảo trì code trong team nhiều người.
Reactive Pattern giúp team làm việc hiệu quả hơn hay tạo thêm độ phức tạp?

👉 Không trả lời chung chung, phải xét:

- junior vs senior
- app nhỏ vs app lớn
- quy ước & kỷ luật code

9 Hiệu năng Flutter khi dùng Stream-based UI

Đánh giá ảnh hưởng của Stream-based UI đến hiệu suất ứng dụng.
Những sai lầm nào trong thiết kế Stream có thể dẫn đến:

- rebuild dư thừa?
- memory leak?
- UI lag dù không crash?

👉 Yêu cầu:

- phân tích nguyên nhân gốc rễ
- không đổ lỗi cho Flutter

10 Thiết kế luồng dữ liệu cho màn hình Flutter phức tạp

Phân tích cách tổ chức luồng dữ liệu cho một màn hình có nhiều nguồn async
(API, local cache, user action, realtime update).
Làm thế nào để kết hợp Stream & Repository mà:

- không làm UI rối?
- không tạo "mạng nhện Stream"?

👉 Gợi ý:

- tách trách nhiệm
- luồng dữ liệu 1 chiều
- tư duy chuẩn bị cho State Management

PhucPT10@fpt.edu.vn

10 PROMPT TỰ NGHIÊN CỨU THỰC TIỄN CHUYÊN NGHIỆP

1 Nhiều StreamBuilder dùng chung một nguồn dữ liệu

Một ứng dụng Flutter có nhiều `StreamBuilder` cùng subscribe một nguồn dữ liệu (ví dụ: user profile, notification count).

❓ **Ở góc độ kiến trúc & performance**, bạn nhận diện được những rủi ro nào?

❓ Bạn sẽ tổ chức lại luồng dữ liệu ra sao để:

- tránh duplicate work
- vẫn giữ UI reactive
- không phá vỡ tính tách lớp?

2 Nhiều màn hình subscribe cùng một Stream từ Repository

Trong dự án lớn, nhiều màn hình subscribe cùng một Stream do Repository cung cấp.

❓ Làm thế nào để kiểm soát **vòng đời Stream** khi:

- màn hình push/pop liên tục?
- app chạy lâu trong background?

❓ Ai chịu trách nhiệm **tạo – giữ – dispose** Stream trong kiến trúc này?

3 Team junior lạm dụng Stream cho dữ liệu tĩnh

Một team junior dùng Stream cho:

- danh sách category tĩnh
- cấu hình app
- dữ liệu chỉ load 1 lần

❓ Với vai trò Tech Lead, bạn sẽ:

- đưa ra **guideline** nào?
- phân biệt **reactive cần thiết** vs **over-engineering** ra sao?

4 UI rebuild quá nhiều trong app realtime

Ứng dụng realtime gặp tình trạng:

- UI rebuild liên tục
- pin tụt nhanh
- FPS giảm

❓ Ở mức kiến trúc, những nguyên nhân nào liên quan đến:

- thiết kế Stream?
- cách dùng StreamBuilder?
- cách tổ chức Repository?

❓ Bạn sẽ tiếp cận **tối ưu từ đâu trước**?

5 Repository tồn tại nhưng UI vẫn "biết quá nhiều"

Dự án có Repository Pattern nhưng:

- Widget vẫn xử lý mapping dữ liệu
- UI hiểu chi tiết API response

❓ Điều này phản ánh vấn đề gì trong kiến trúc?

❓ Bạn sẽ refactor theo hướng nào để Repository **thực sự che chắn UI khỏi data source**?

6 Debug Stream trong production trở nên khó khăn

Khi số lượng Stream tăng:

- log khó đọc
- lỗi xảy ra ngẫu nhiên
- stack trace không rõ nguồn

❓ Bạn sẽ thiết kế:

PhucPT10@fpt.edu.vn

- logging strategy
- error handling
- monitoring

cho Stream-based app như thế nào trong môi trường production?

7 Stream vẫn chạy khi người dùng đã rời màn hình

Một Stream vẫn tiếp tục phát dữ liệu dù người dùng đã rời màn hình từ lâu.

❓ Nếu không dispose đúng cách, **hậu quả dài hạn** có thể là gì?

❓ Làm sao để kiến trúc **không phụ thuộc vào việc dev "nhớ dispose"**?

8 Reactive Pattern vs độ phức tạp của code

Reactive Pattern giúp UI đồng bộ tốt hơn nhưng:

- code khó đọc hơn
- onboarding dev mới mất thời gian

❓ Là Tech Lead, bạn sẽ:

- cho phép reactive tới mức nào?
- đặt ranh giới ra sao giữa **clarity** và **reactivity**?

9 Một màn hình phụ thuộc nhiều Stream từ nhiều Repository

Một màn hình cần:

- user info
- permission
- notification
- realtime data

❓ Bạn sẽ thiết kế luồng dữ liệu thế nào để:

- UI không trở thành "bãi rác logic"
- mỗi layer vẫn giữ đúng trách nhiệm?

10 Business thay đổi liên tục → Stream-based code khó chỉnh sửa

Khi yêu cầu business đổi liên tục, code dùng Stream dễ bị:

- chắp vá
- phụ thuộc chéo
- technical debt tăng nhanh

❓ Những **nguyên tắc thiết kế ban đầu** nào giúp giảm chi phí refactor về sau?

11 Scale app: setState vs Reactive Pattern

So sánh hai dự án:

- App A: chủ yếu dùng `setState`
- App B: dùng Reactive Pattern + Repository

❓ Ở quy mô team 5–10 người, sự khác biệt thể hiện rõ nhất ở:

- tốc độ phát triển?
- bug?
- khả năng review code?

12 Bug chỉ xuất hiện khi dữ liệu Stream đến quá nhanh

Một bug chỉ xảy ra khi:

- dữ liệu Stream đến dồn dập
- không tái hiện được ở local

❓ Bạn sẽ tiếp cận debug thế nào trong production?

❓ Cần quan sát **luồng dữ liệu** hay **UI state** trước?

PhucPT10@fpt.edu.vn

13 Repository trả về Stream trực tiếp từ data source

Repository hiện tại chỉ "pass-through" Stream từ API / database.

❓ Trong dự án lớn, có nên:

- che giấu Stream bên trong Repository?
- expose Stream ra ngoài?

❓ Quyết định này ảnh hưởng gì đến:

- test
- refactor
- thay đổi data source?

14 Refactor Stream gây ripple effect

Khi thay đổi một Stream:

- nhiều màn hình bị ảnh hưởng
- sửa một chỗ vỡ nhiều chỗ

❓ Bạn sẽ tổ chức code thế nào để:

- giảm ripple effect
- cô lập thay đổi?

15 Rule cho team khi dùng Stream & StreamBuilder

Nếu bạn là Tech Lead, hãy đề xuất:

- các **rule bắt buộc**
- các **anti-pattern bị cấm**

khi team sử dụng Stream & StreamBuilder để đảm bảo:

- maintainability
- performance
- codebase sống lâu dài

.

PhucPT10@fpt.edu.vn

PhucPT10@fpt.edu.vn